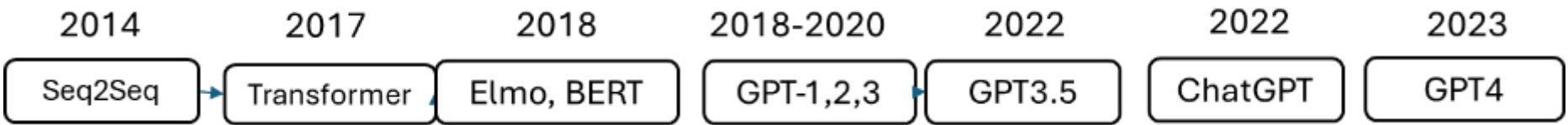


# DASC7606 - 10

## **Deep Learning**

**DeepSeek, Text2Image**  
**Deep Reinforcement Learning**

Dr. Bethany Chan  
Professor Francis Chin  
2025



**Hangzhou DeepSeek Artificial Intelligence Basic Technology Research Co., Ltd.**



|                     |                                                   |
|---------------------|---------------------------------------------------|
| Native name         | 杭州深度求索人工智能基础技术研究有限公司                              |
| Company type        | Private                                           |
| Industry            | Information technology<br>Artificial intelligence |
| Founded             | May 2023                                          |
| Founder             | Liang Wenfeng                                     |
| Headquarters        | Hangzhou, Zhejiang, China                         |
| Key people          | Liang Wenfeng (CEO)                               |
| Owner               | High-Flyer                                        |
| Number of employees | Under 200                                         |
| Website             | <a href="https://deepseek.com">deepseek.com</a>   |

|               |                                                          |
|---------------|----------------------------------------------------------|
| Feb 2016:     | High-Flyer co-founded by Liang Wenfeng                   |
| 2019-2021:    | High-Flyer developed as a hedge fund using AI in trading |
| Apr/May 2023: | High-Flyer started DeepSeek for AGI                      |
| Nov 2023:     | DeepSeek-Coder                                           |
| Nov 2023:     | DeepSeek-LLM (7B and 67B) and chat versions              |
| Jan 2024:     | DeepSeek-MoE (16B)                                       |
| Jan 2024:     | DeepSeek-MoE paper                                       |
| Apr 2024:     | DeepSeek-Math                                            |
| Apr 2024:     | Group Relative Policy Optimization (GRPO) paper          |
| May 2024:     | DeepSeek-V2                                              |
| Sep 2024:     | DeepSeek-V2.5                                            |
| Dec 2024:     | DeepSeek-V3                                              |
| Jan 2025:     | DeepSeek-R1                                              |

## LLM Training Process (in general)



### Pre-training

- Start with an architecture
- Pre-train on a big dataset
- Goal: to be good at next token/word prediction
- ▶ *Costly when architecture has lots of parameters and dataset is huge*
- ▶ *Not reliable at following instructions: prompt does not trigger what you want*

### Supervised fine-tuning (SFT)

- Teach the LLM to follow instructions
- Train on dataset of (instruction, response)
- ▶ *May want to use LoRA*

### Reinforcement learning (RL)

- Teach the LLM to do better with feedback:
  - Human feedback (HF)
  - AI feedback
- ▶ *Human feedback can be costly*

## DeepSeek-V3 (the chat model)

### Pre-training

- Start with 671B parameter **MoE** architecture
- Trained on 14.8T tokens: multilingual corpus, mostly English and Chinese and higher ratio of math and coding data
- Extend context length from 4K to 32K to 128K, using YaRN
- Resulting in **DeepSeek-V3-Base** (starting point for DeepSeek-R1-Zero)
- Cost: US\$5.328m

### Supervised fine-tuning (SFT)

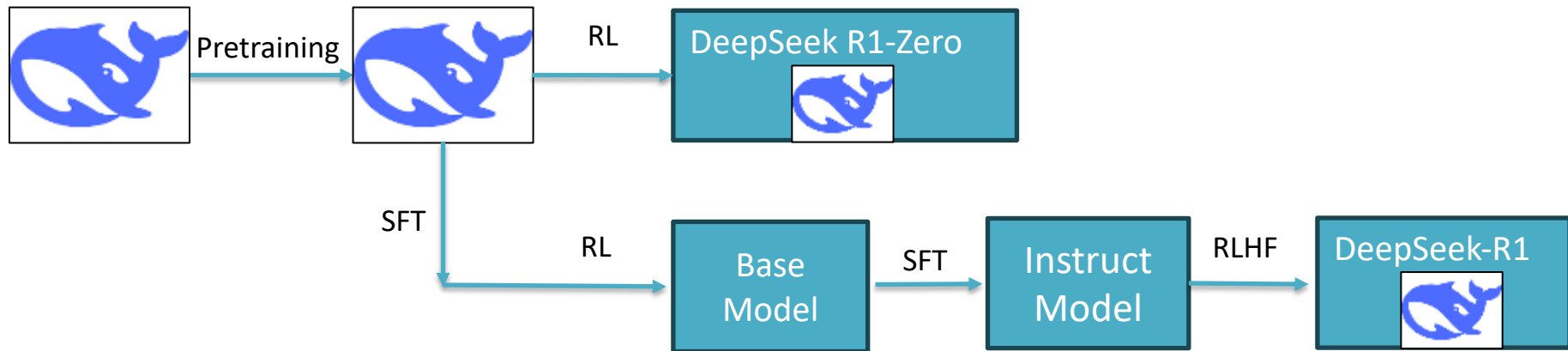
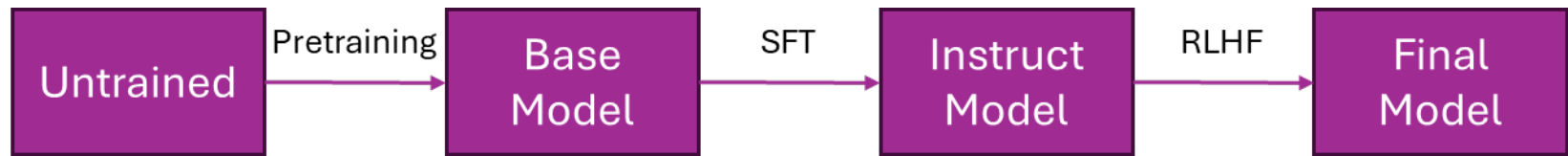
- Trained expert models to generate samples of reasoning data (math, programming, logic) and used V2.5 to generate non-reasoning data
- Trained DeepSeek-V3-Base with the 1.5M samples for 2 epochs
- Resulting in “SFT checkpoint of V3”

### Reinforcement learning (RL)

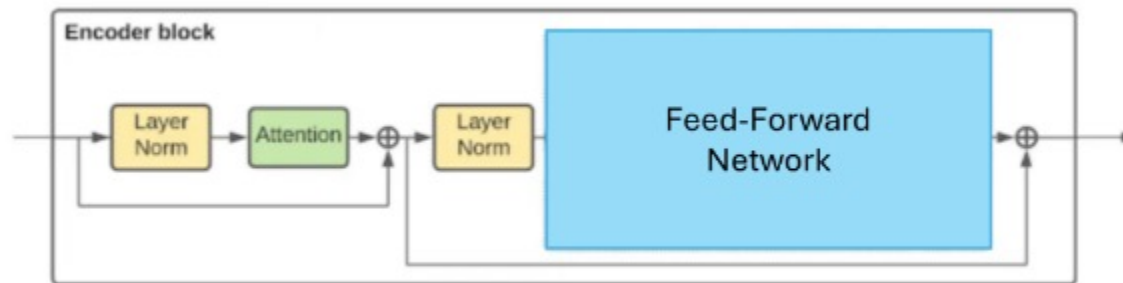
- Created reward model starting from SFT checkpoint of V3
- Train SFT checkpoint of V3 with **GRPO** using both reward model and **rule-based reward**
- Resulting in DeepSeek-V3

Total training cost: < US\$5.6m

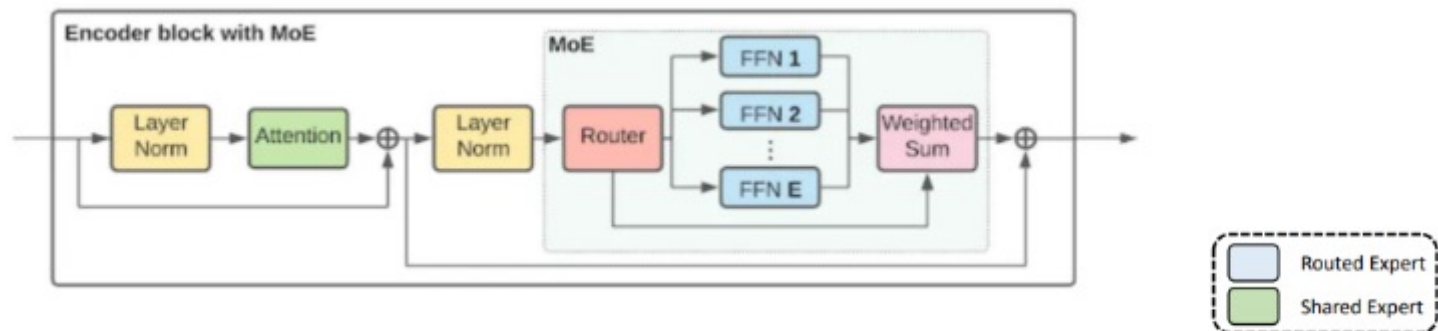
# Training pipeline



Example of  
FFN in  
transformer



Mixture-of-Experts (MoE)  
replaces FFN



Explanation of  
DeepSeek's MoE  
architecture:

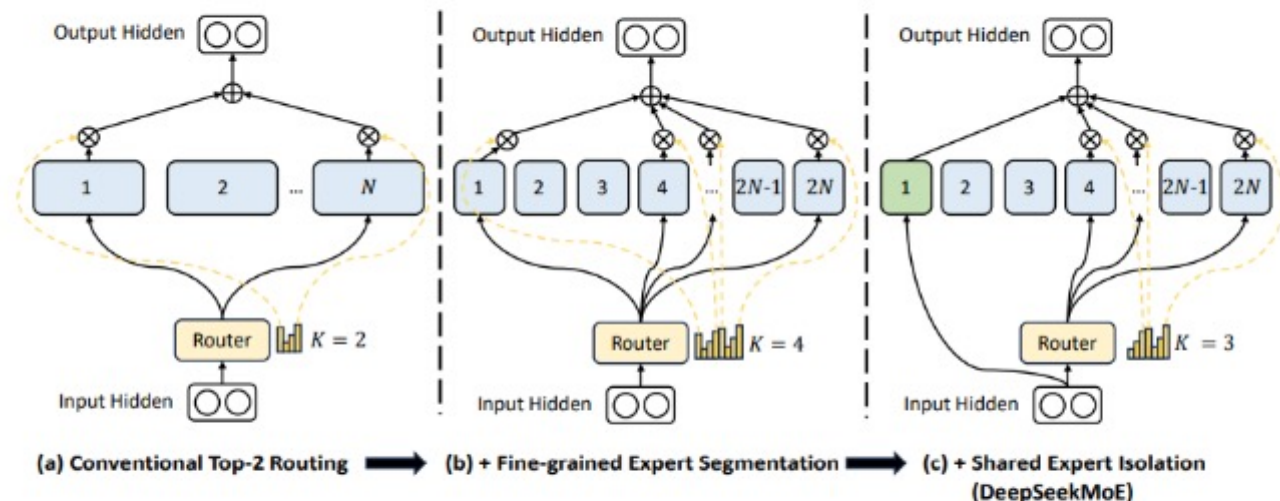


Figure 2 | Illustration of DeepSeekMoE. Subfigure (a) showcases an MoE layer with the conventional top-2 routing strategy. Subfigure (b) illustrates the fine-grained expert segmentation strategy. Subsequently, subfigure (c) demonstrates the integration of the shared expert isolation strategy, constituting the complete DeepSeekMoE architecture. It is noteworthy that across these three architectures, the number of expert parameters and computational costs remain constant.

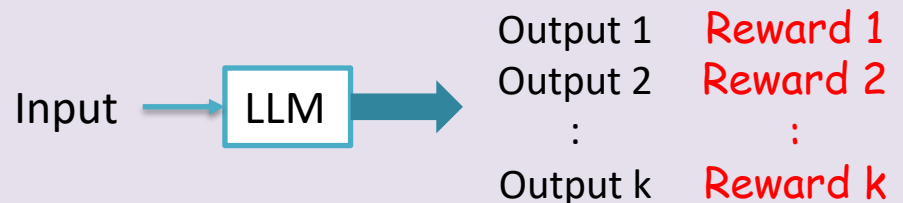
## DeepSeek-R1-Zero (analogy with Alpha Go Zero – pure RL)

### Pre-training

- Start with 671B parameter DeepSeek-V3-Base which was pretrained on 14.8T tokens: mostly English and Chinese and higher ratio of math and coding data

### Reinforcement learning (RL)

- Group Relative Policy Optimization (GRPO)



- Rule-based reward
  - Accuracy
  - Format

---

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>` `</think>` and `<answer>` `</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: **prompt**. Assistant:

---

Table 1 | Template for DeepSeek-R1-Zero. **prompt** will be replaced with the specific reasoning question during training.

| Model            | AIME 2024 |         | MATH-500 |
|------------------|-----------|---------|----------|
|                  | pass@1    | cons@64 | pass@1   |
| OpenAI-o1-mini   | 63.6      | 80.0    | 90.0     |
| OpenAI-o1-0912   | 74.4      | 83.3    | 94.8     |
| DeepSeek-R1-Zero | 71.0      | 86.7    | 95.9     |

## DeepSeek-R1 (to solve (a) readability issues (b) language mixing problem)

### **[1] SFT**

- Start with DeepSeek-V3-Base
- Train on high-quality readable results from DeepSeek-R1-Zero

*Fine-tune for readability*

### **[2] RL for reasoning**

- GRPO on reasoning problems (math, programming, logic) with rule-based rewards

*Reinforcing good reasoning patterns*

### **[3] SFT**

- Use model from [2] to generate lots of reasoning samples and use DeepSeek-V3 to retain only correct and readable output
- Model from [2] is fine-tune trained on retained samples and some data from DeepSeek-V3's training data so that the model is exposed to non-reasoning data

*Fine-tune for good reasoning and non-reasoning capabilities*

### **[4] RL for diverse tasks**

- GRPO on diverse tasks with rule-based rewards used where appropriate and LLM feedback to align with human preferences otherwise

*Reinforce for all scenarios*

| Benchmark (Metric) |                            | Claude-3.5-<br>Sonnet-1022 | GPT-4o<br>0513 | DeepSeek<br>V3 | OpenAI<br>o1-mini | OpenAI<br>o1-1217 | DeepSeek<br>R1 |
|--------------------|----------------------------|----------------------------|----------------|----------------|-------------------|-------------------|----------------|
| Architecture       |                            | -                          | -              | MoE            | -                 | -                 | MoE            |
| # Activated Params |                            | -                          | -              | 37B            | -                 | -                 | 37B            |
| # Total Params     |                            | -                          | -              | 671B           | -                 | -                 | 671B           |
| English            | MMLU (Pass@1)              | 88.3                       | 87.2           | 88.5           | 85.2              | <b>91.8</b>       | 90.8           |
|                    | MMLU-Redux (EM)            | 88.9                       | 88.0           | 89.1           | 86.7              | -                 | <b>92.9</b>    |
|                    | MMLU-Pro (EM)              | 78.0                       | 72.6           | 75.9           | 80.3              | -                 | <b>84.0</b>    |
|                    | DROP (3-shot F1)           | 88.3                       | 83.7           | 91.6           | 83.9              | 90.2              | <b>92.2</b>    |
|                    | IF-Eval (Prompt Strict)    | <b>86.5</b>                | 84.3           | 86.1           | 84.8              | -                 | 83.3           |
|                    | GPQA Diamond (Pass@1)      | 65.0                       | 49.9           | 59.1           | 60.0              | <b>75.7</b>       | 71.5           |
|                    | SimpleQA (Correct)         | 28.4                       | 38.2           | 24.9           | 7.0               | <b>47.0</b>       | 30.1           |
|                    | FRAMES (Acc.)              | 72.5                       | 80.5           | 73.3           | 76.9              | -                 | <b>82.5</b>    |
|                    | AlpacaEval2.0 (LC-winrate) | 52.0                       | 51.1           | 70.0           | 57.8              | -                 | <b>87.6</b>    |
|                    | ArenaHard (GPT-4-1106)     | 85.2                       | 80.4           | 85.5           | 92.0              | -                 | <b>92.3</b>    |
| Code               | LiveCodeBench (Pass@1-COT) | 38.9                       | 32.9           | 36.2           | 53.8              | 63.4              | <b>65.9</b>    |
|                    | Codeforces (Percentile)    | 20.3                       | 23.6           | 58.7           | 93.4              | <b>96.6</b>       | 96.3           |
|                    | Codeforces (Rating)        | 717                        | 759            | 1134           | 1820              | <b>2061</b>       | 2029           |
|                    | SWE Verified (Resolved)    | <b>50.8</b>                | 38.8           | 42.0           | 41.6              | 48.9              | 49.2           |
|                    | Aider-Polyglot (Acc.)      | 45.3                       | 16.0           | 49.6           | 32.9              | <b>61.7</b>       | 53.3           |
| Math               | AIME 2024 (Pass@1)         | 16.0                       | 9.3            | 39.2           | 63.6              | 79.2              | <b>79.8</b>    |
|                    | MATH-500 (Pass@1)          | 78.3                       | 74.6           | 90.2           | 90.0              | 96.4              | <b>97.3</b>    |
|                    | CNMO 2024 (Pass@1)         | 13.1                       | 10.8           | 43.2           | 67.6              | -                 | <b>78.8</b>    |
| Chinese            | CLUEWSC (EM)               | 85.4                       | 87.9           | 90.9           | 89.9              | -                 | <b>92.8</b>    |
|                    | C-Eval (EM)                | 76.7                       | 76.0           | 86.5           | 68.9              | -                 | <b>91.8</b>    |
|                    | C-SimpleQA (Correct)       | 55.4                       | 58.7           | <b>68.0</b>    | 40.3              | -                 | 63.7           |

Table 4 | Comparison between DeepSeek-R1 and other representative models.

## Distillation of DeepSeek-R1

| Model                         | AIME 2024   |             | MATH-500    | GPQA<br>Diamond | LiveCode<br>Bench | CodeForces  |
|-------------------------------|-------------|-------------|-------------|-----------------|-------------------|-------------|
|                               | pass@1      | cons@64     | pass@1      | pass@1          | pass@1            | rating      |
| GPT-4o-0513                   | 9.3         | 13.4        | 74.6        | 49.9            | 32.9              | 759         |
| Claude-3.5-Sonnet-1022        | 16.0        | 26.7        | 78.3        | 65.0            | 38.9              | 717         |
| OpenAI-o1-mini                | 63.6        | 80.0        | 90.0        | 60.0            | 53.8              | <b>1820</b> |
| QwQ-32B-Preview               | 50.0        | 60.0        | 90.6        | 54.5            | 41.9              | 1316        |
| DeepSeek-R1-Distill-Qwen-1.5B | 28.9        | 52.7        | 83.9        | 33.8            | 16.9              | 954         |
| DeepSeek-R1-Distill-Qwen-7B   | 55.5        | 83.3        | 92.8        | 49.1            | 37.6              | 1189        |
| DeepSeek-R1-Distill-Qwen-14B  | 69.7        | 80.0        | 93.9        | 59.1            | 53.1              | 1481        |
| DeepSeek-R1-Distill-Qwen-32B  | <b>72.6</b> | 83.3        | 94.3        | 62.1            | 57.2              | 1691        |
| DeepSeek-R1-Distill-Llama-8B  | 50.4        | 80.0        | 89.1        | 49.0            | 39.6              | 1205        |
| DeepSeek-R1-Distill-Llama-70B | 70.0        | <b>86.7</b> | <b>94.5</b> | <b>65.2</b>     | <b>57.5</b>       | 1633        |

Table 5 | Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks.

For the distillation, SFT only on 800k samples

### DeepSeek-R1-Zero

**Pre-Training → ~~Supervised Fine-Tuning~~ → Reinforcement Learning**

- We directly apply RL to the base model without relying on supervised fine-tuning (SFT) as a preliminary step. This approach allows the model to explore chain-of-thought (CoT) for solving complex problems, resulting in the development of DeepSeek-R1-Zero. DeepSeek-R1-Zero demonstrates capabilities such as self-verification, reflection, and generating long CoTs, marking a significant milestone for the research community. Notably, it is the first open research to validate that reasoning capabilities of LLMs can be incentivized purely through RL, without the need for SFT. This breakthrough paves the way for future advancements in this area.

### DeepSeek-R1

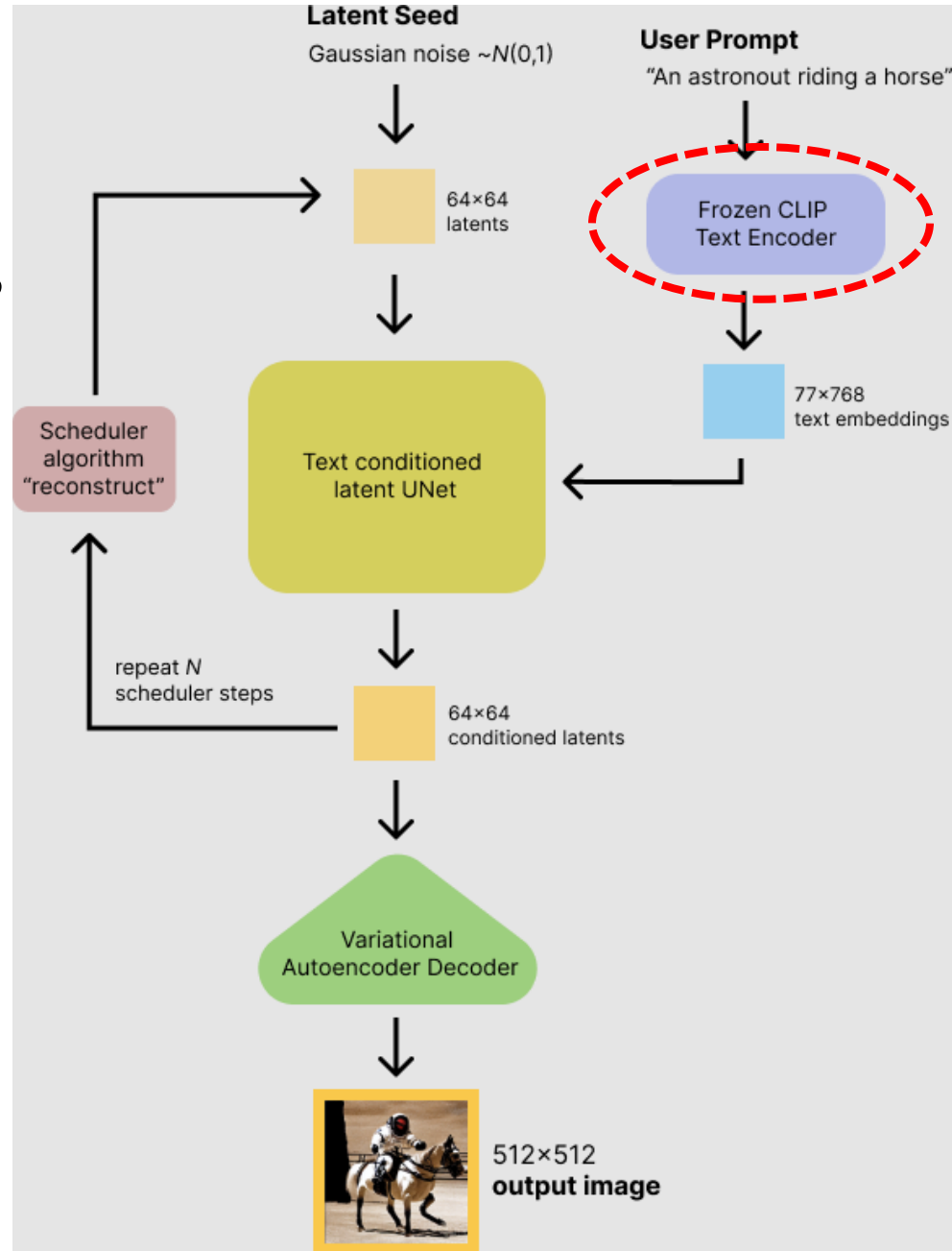
**Pre-Training → Supervised Fine-Tuning → Reinforcement Learning  
→ Supervised Fine-Tuning → Reinforcement Learning**

- We introduce our pipeline to develop DeepSeek-R1. The pipeline incorporates two RL stages aimed at discovering improved reasoning patterns and aligning with human preferences, as well as two SFT stages that serve as the seed for the model's reasoning and non-reasoning capabilities. We believe the pipeline will benefit the industry by creating better models.

**Text 2 Image**

# Text 2 Image

- **Diffusion Model:**  
Control how the image looks through a process called ***conditioning*** during this reverse diffusion process
- Text conditioning works by passing the text embeddings to the U-Net, i.e., merges the text embedding with the results from every step of the reverse diffusion.
- **Text encoder: CLIP**



# CLIP - *Contrastive Language-Image Pretraining*

<https://openai.com/research/clip>

<https://towardsdatascience.com/clip-the-most-influential-ai-model-from-openai-and-how-to-use-it-f8ee408958b1>

## An open source, multi-modal, zero-shot model

- Contrastive - between similar and dissimilar objects, i.e., keep latent representations of similar object close while dissimilar object's latent representations far away.
- Open source by OpenAI
- Multi-model – natural language and computer vision
- Zero-shot – unseen labels, without specifically trained
- Trained on 400 million image-text (available on internet)

CLIP finds the best text prompt describing the image.

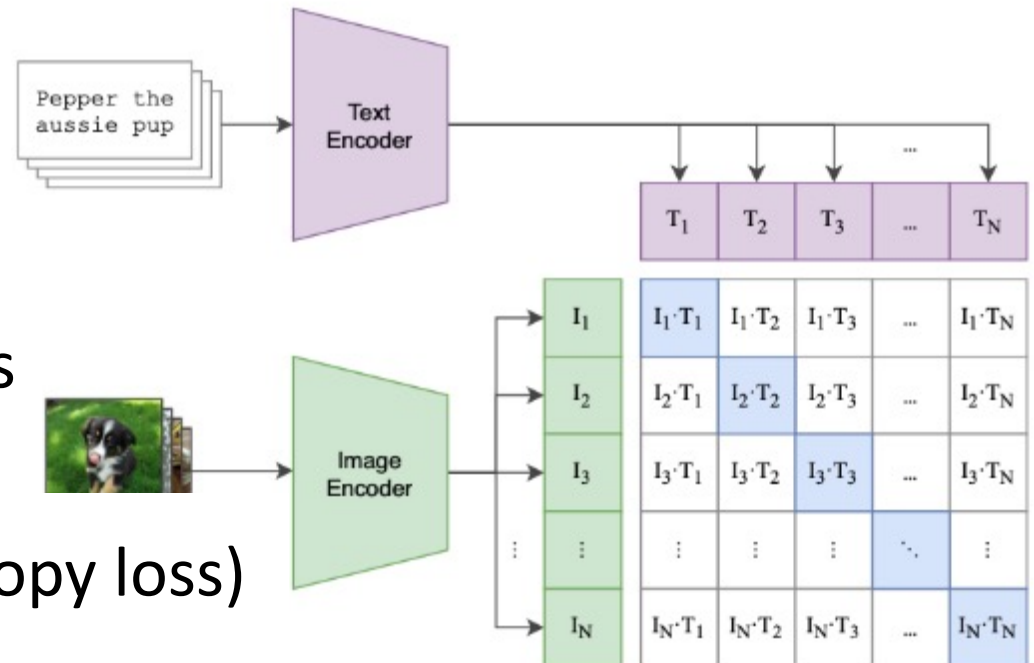


| Prompts                 | Probabilities |
|-------------------------|---------------|
| A girl wearing a beanie | 90.05%        |
| A girl wearing a hat    | 9.33%         |
| A boy wearing a beanie  | 0.57%         |
| A girl riding a bike    | 0.04%         |
| A dog                   | 0.00%         |

# Contrastive Pre-training

- Training data –  $N$  <image-text> pairs  
 $\{ \langle \text{image1}, \text{text1} \rangle, \dots \langle \text{imageN}, \text{textN} \rangle \}$ 
  - Caption is superior to manual label - labelling number of cars and dogs in picture cannot give a feel for the scene overall.
- Compute text encoding (based on GPT2) and image encoding (based on ResNet) of size  $d$
- Train both encoders on  $N \times N$  cosine similarities such that matched image-text pairs (diagonals) are **maximized** and the non-matched pairs (off-diagonals) are **minimized**.  
(symmetric cross-entropy loss)

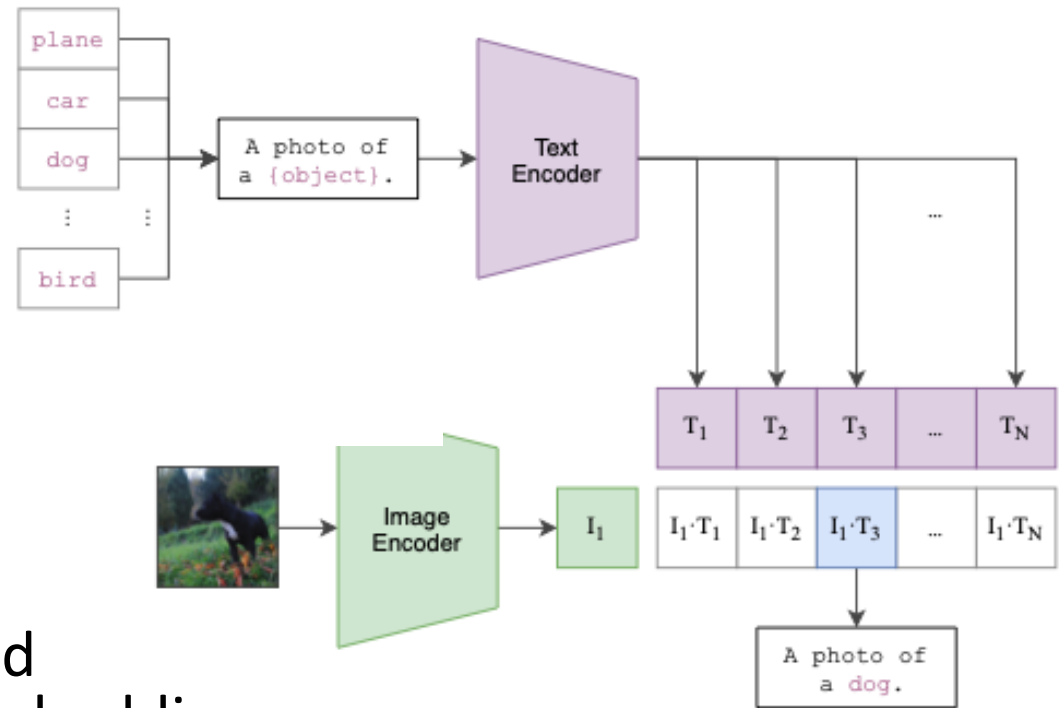
(1) Contrastive pre-training



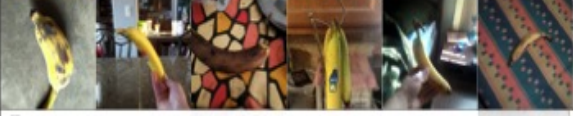
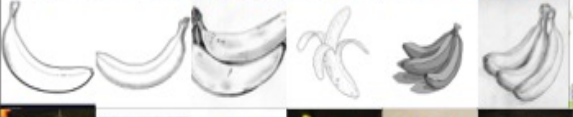
# Application: Zero-shot Classification

- K-shot learning means training set contains K instances of each class.
  - Few-shot means  $K < 10$
  - One-shot when  $K = 1$
  - Zero-shot when  $K = 0$
- Provide a set of text, “a photo of a dog” or “a photo of a cat” ... one for each class
- Encode each text into text embedding
- The image to be classified is encoded into image embedding
- **CLIP** computes the pairwise cosine similarities between the image and the text embeddings. The text prompt with the highest similarity is chosen as the prediction.

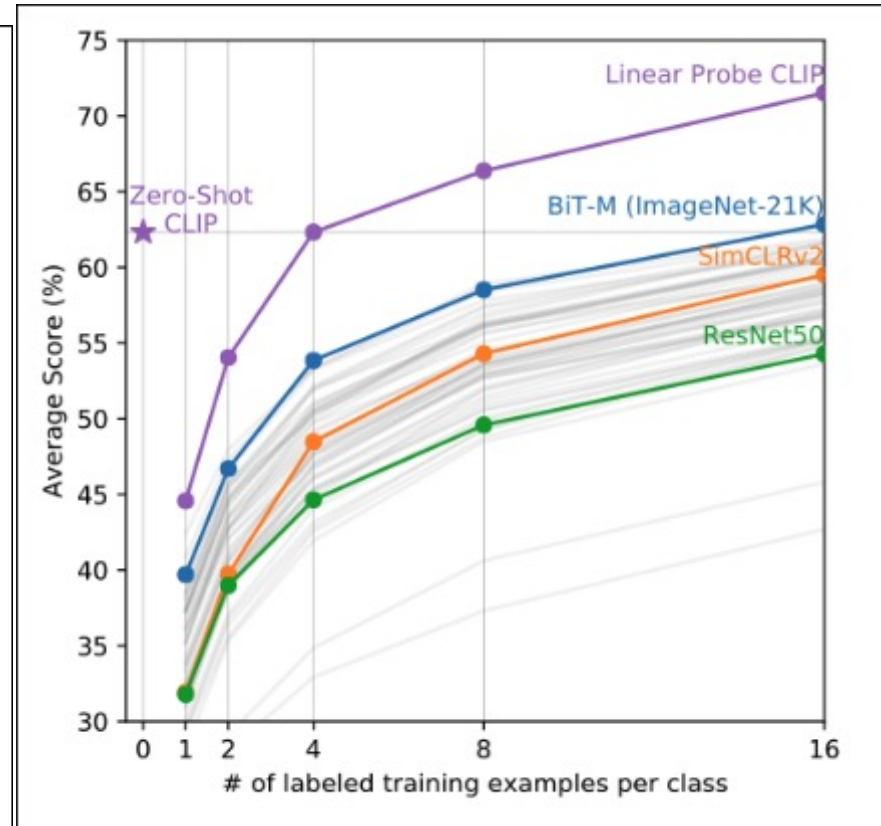
(2) Create dataset classifier from label text



# Impact of CLIP

|                 | Dataset Examples                                                                   | ImageNet<br>ResNet101 | Zero-Shot<br>CLIP | $\Delta$ Score |
|-----------------|------------------------------------------------------------------------------------|-----------------------|-------------------|----------------|
| ImageNet        |   | 76.2                  | 76.2              | 0%             |
| ImageNetV2      |   | 64.3                  | 70.1              | +5.8%          |
| ImageNet-R      |   | 37.7                  | 88.9              | +51.2%         |
| ObjectNet       |   | 32.6                  | 72.3              | +39.7%         |
| ImageNet Sketch |   | 25.2                  | 60.2              | +35.0%         |
| ImageNet-A      |  | 2.7                   | 77.1              | +74.4%         |

Robustness to Distribution Shift  
i.e., Data model changes over time



Significantly outperforms  
other classifiers on few-shot  
classification

# **Deep Reinforcement Learning**

# Reinforcement Learning (RL) vs Supervised Learning

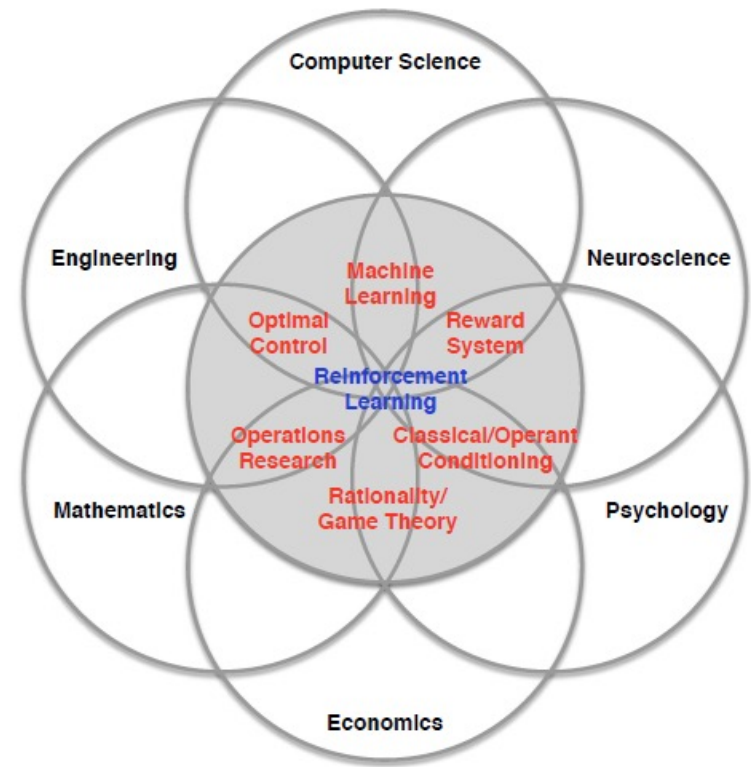
**Supervised learning** – Learning from labeled data

Supervised learning is **learning from a labeled training dataset** and then applying that learning on a new testing dataset to make predictions

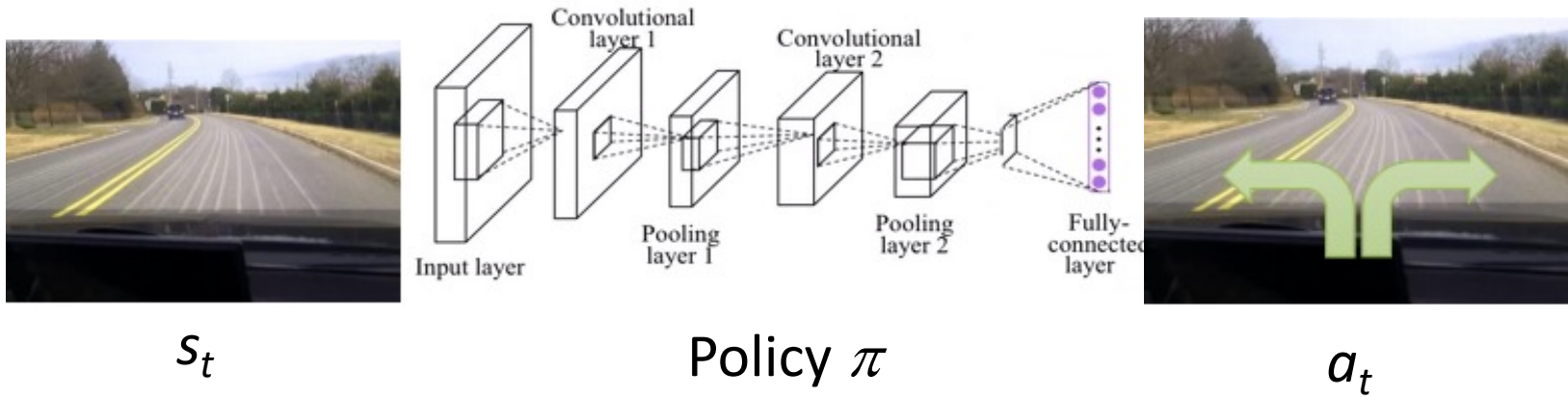
**Reinforcement learning** –

Trial-and-error learning from feedback while doing (e.g. bicycle riding)

Reinforcement learning is **dynamically learning** by adjusting actions **based on feedback** to maximize a reward



# Reinforcement Learning (RL) vs Supervised Learning



## Reinforcement Learning:

- trial-and-error learning
- where agent finds a good policy  $\pi$
- from experiences with environment
- without losing much reward along the way

# Policies: Deterministic vs Stochastic

- Deterministic policy:  $\pi(s)$

$\pi$  maps state to action

$$\pi: S \rightarrow A$$

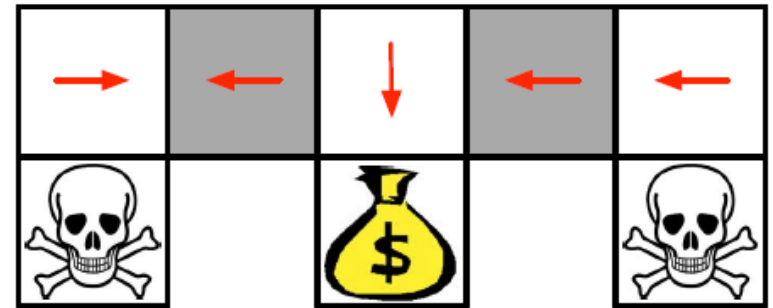
- Stochastic policy:  $\pi(a|s)$

$\pi$  maps states to probability distribution for actions

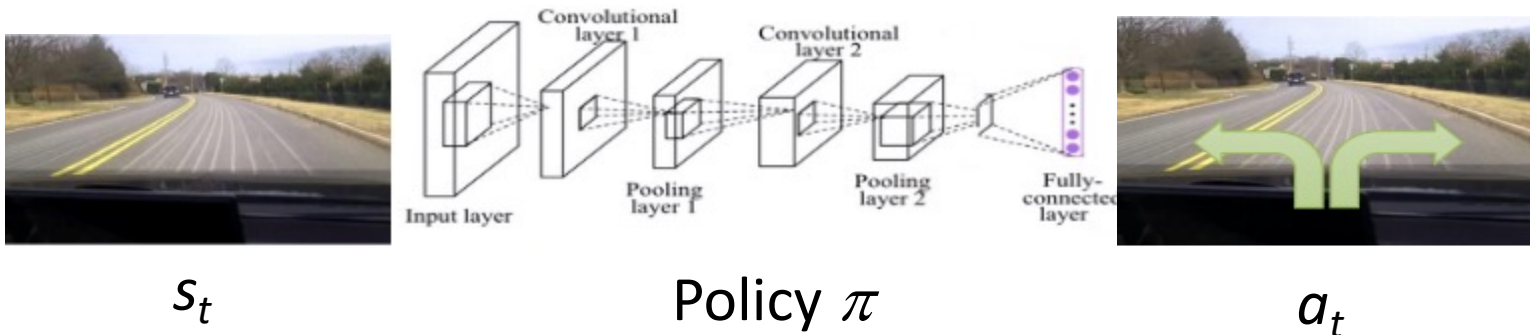
$$\pi: S \rightarrow P(A)$$

- Parameterized policy:  $\pi_\theta$

Objective: get  avoid 



- Cannot distinguish between grey cells
- Deterministic policy gets stuck



# Q-Learning Algorithm [Watkins, 1989]

$Q^*(s_t, a_t)$  = Expected maximum future cumulative reward starting at state  $s_t$  and taking action  $a_t$  following best policy  $\pi$

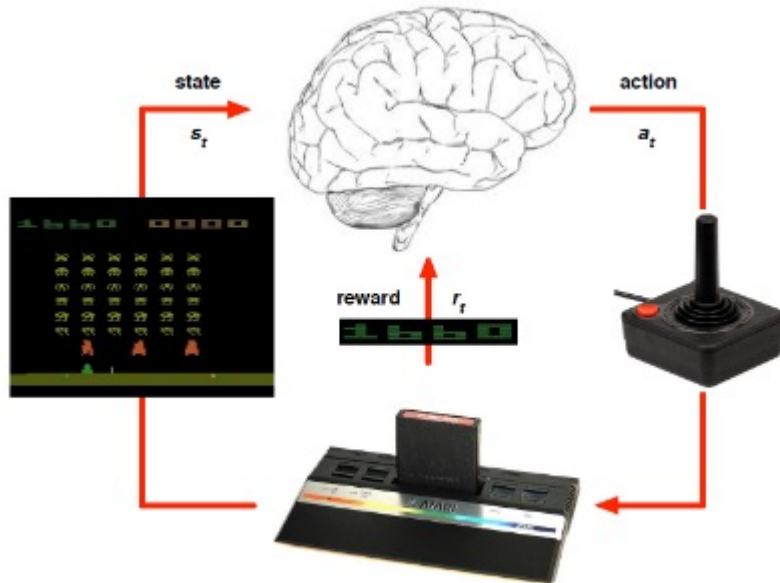
1. Initialize all  $Q(s, a)$  arbitrarily
2. for each training episode do
3.   Initialize state  $s$
4.   while  $s$  is not the terminal state do
5.     Choose action  $a$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
6.     Take action  $a$ , observe reward  $r$  and new state  $s'$
7.      $Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$
8.      $s \leftarrow s'$
9.   end
10. end

Algorithm works by **successively improving** its evaluations of  $Q(s, a)$ , the quality of particular actions at particular states.

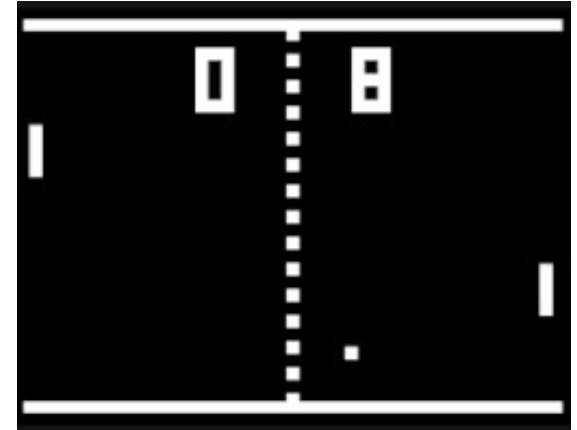
# $\epsilon$ -greedy: Exploration and Exploitation

- **Exploration** – find more information about the environment
- **Exploitation** – exploits known information to maximise reward
- IMPORTANT TO EXPLORE AS WELL AS EXPLOIT
- Restaurant Selection
  - **Exploitation**: Go to your favourite restaurant
  - **Exploration**: Try a new restaurant
- Online Banner Advertisements
  - **Exploitation**: Show the most successful ad
  - **Exploration**: Show a different ad
- Balance between **exploration** and **exploitation**
- **Exploitation**: with probability  $1-\epsilon$  use greedy action, i.e.,  
$$a_t = \arg \max_a Q(s_t, a)$$
- **Exploration**: with probability  $\epsilon$  use random action

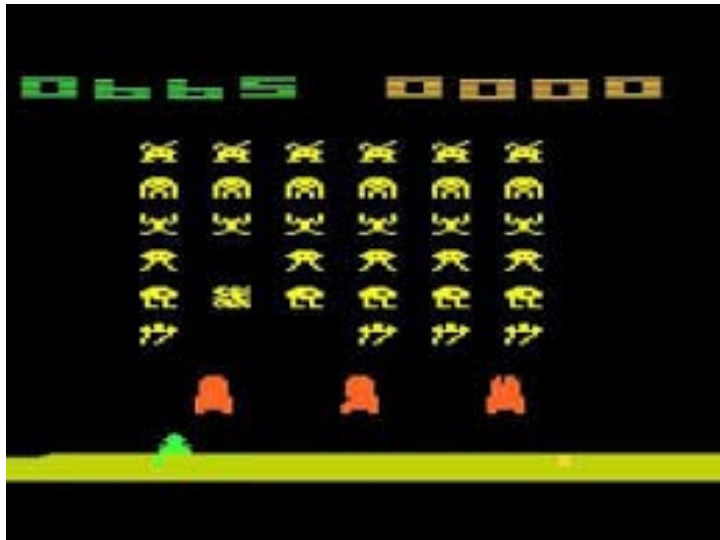
# DQN with Experience Replay applied to Atari games



Atari Pong (introduced in 1972)



Atari Space Invaders

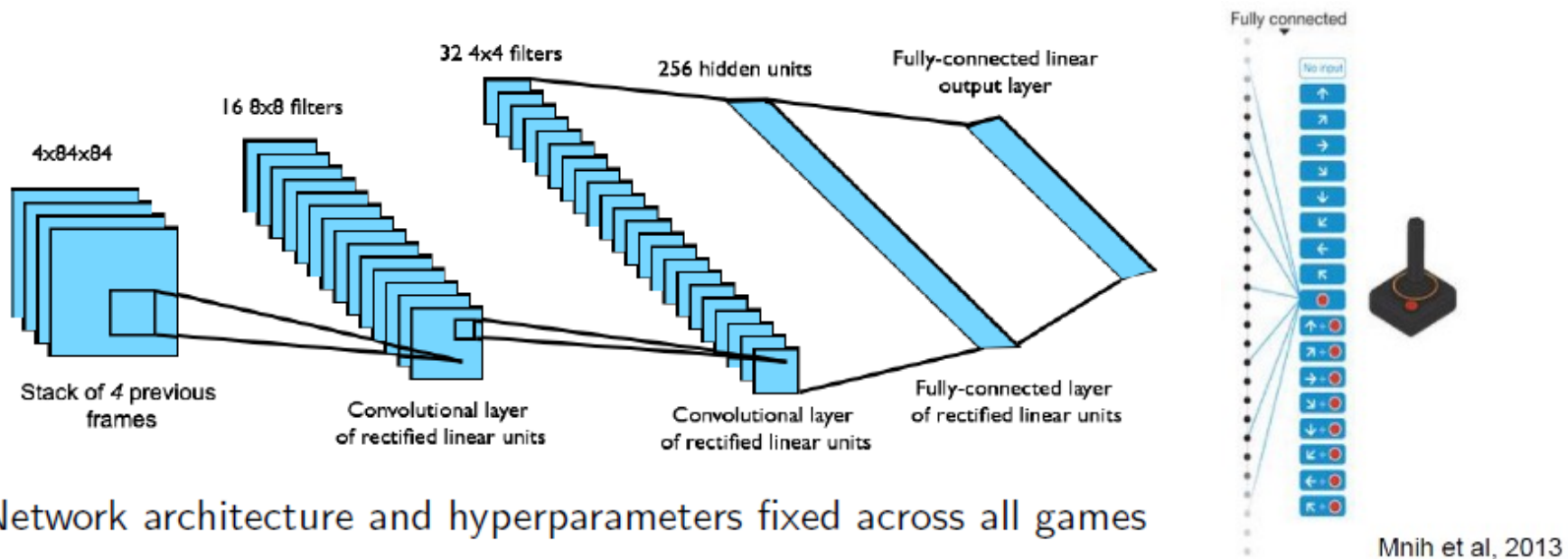


Atari Breakout



# DQN in Atari

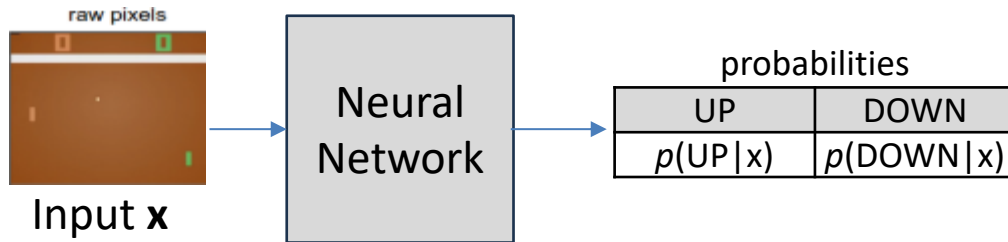
- ▶ End-to-end learning of values  $Q(s, a)$  from pixels  $s$
- ▶ Input state  $s$  is stack of raw pixels from last 4 frames
- ▶ Output is  $Q(s, a)$  for 18 joystick/button positions
- ▶ Reward is change in score for that step



Network architecture and hyperparameters fixed across all games

## Training

### FORWARD PASS

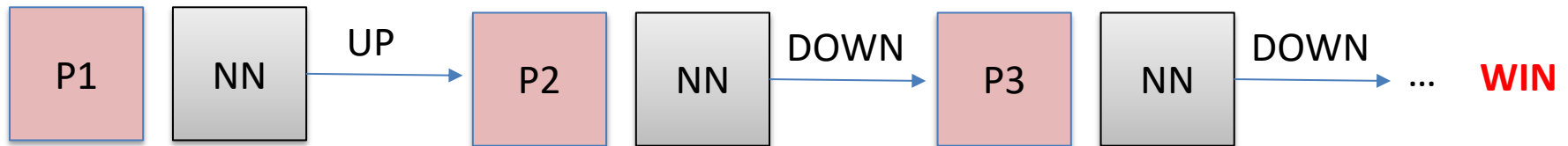


### BACKWARD PASS: how to adjust weights

## Revisit: RL vs supervised learning

Supervised learning: Given supervised learning data  $(x_1, y_1), (x_2, y_2), \dots$

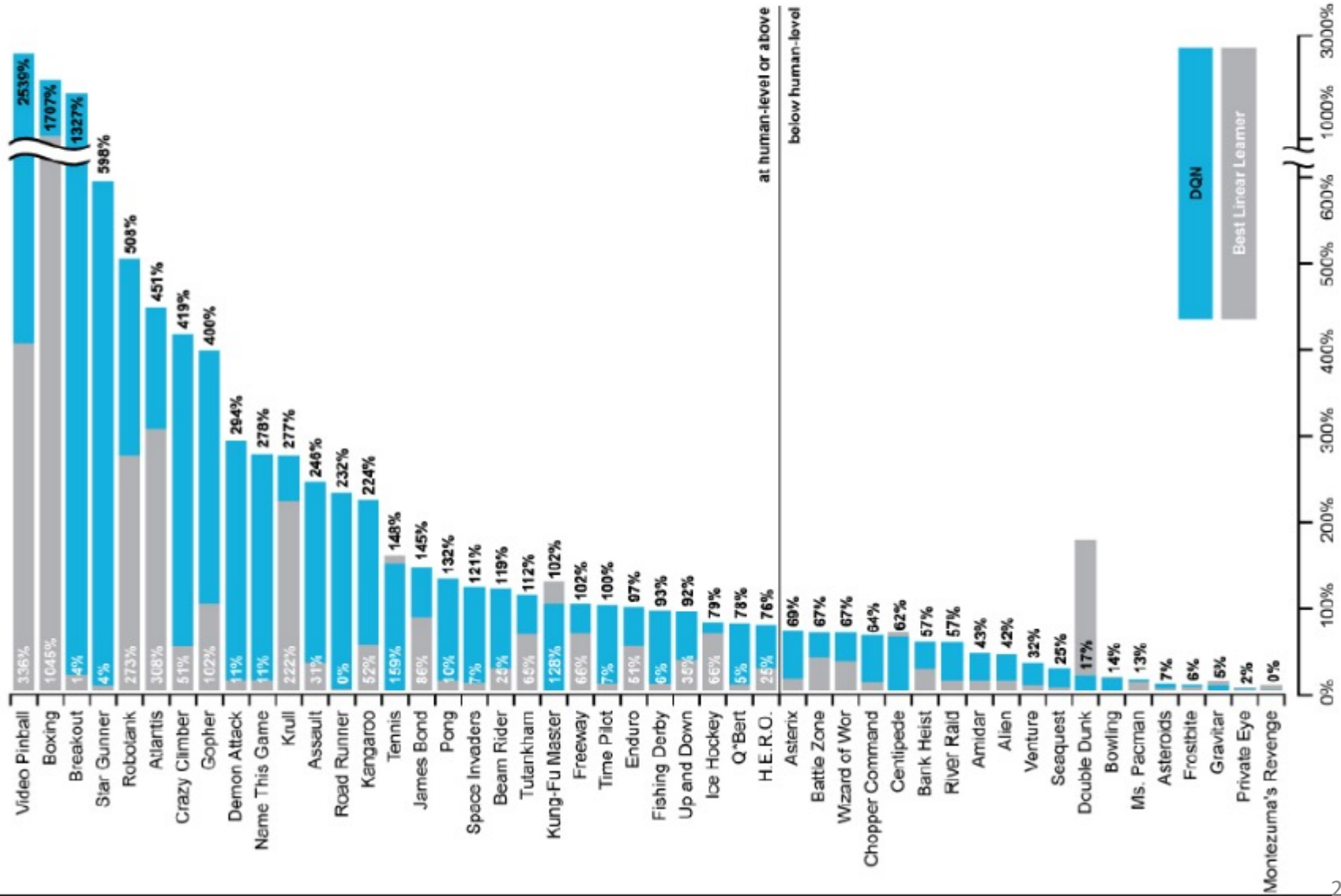
Reinforcement learning: the correct label is given to you later...



RL training data:  $(P1, \text{UP}), (P2, \text{DOWN}), (P3, \text{DOWN}), \dots$

# DQN Results in Atari

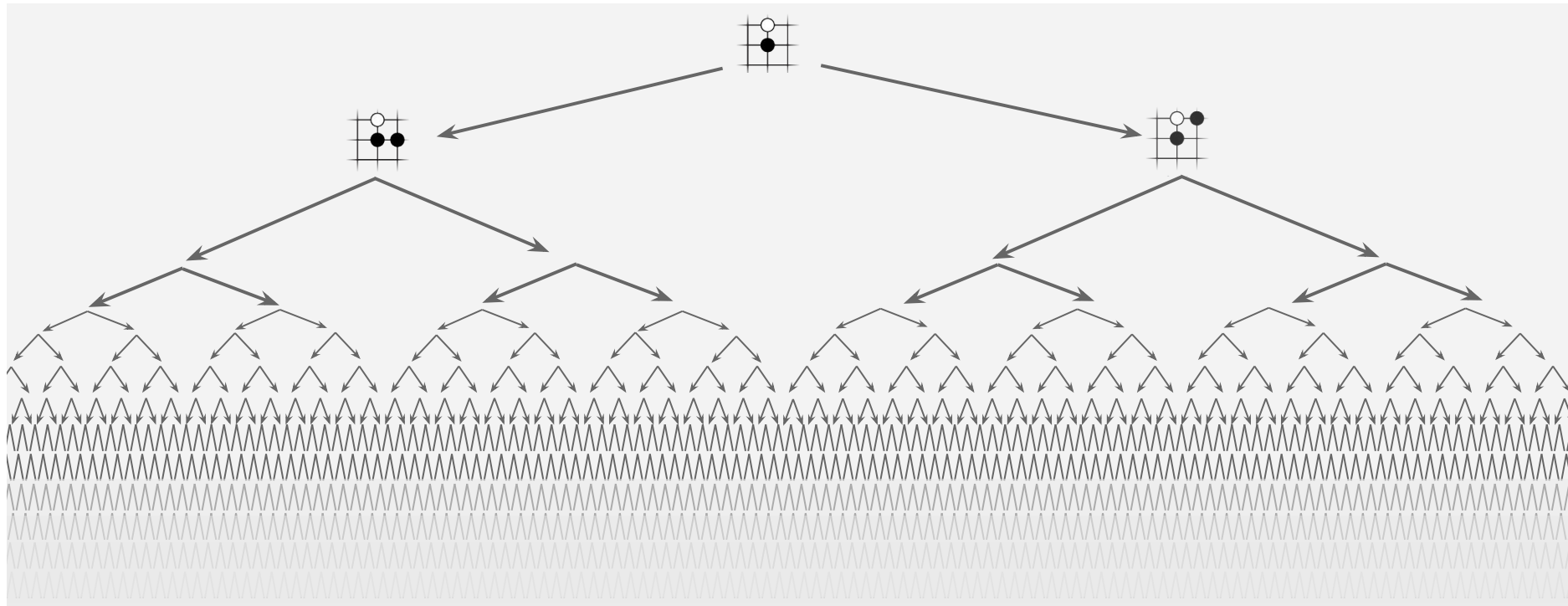
Mnih, V., Kavukcuoglu, K., Silver, D. *et al.* Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015). <https://doi.org/10.1038/nature14236>



# AlphaGo\*

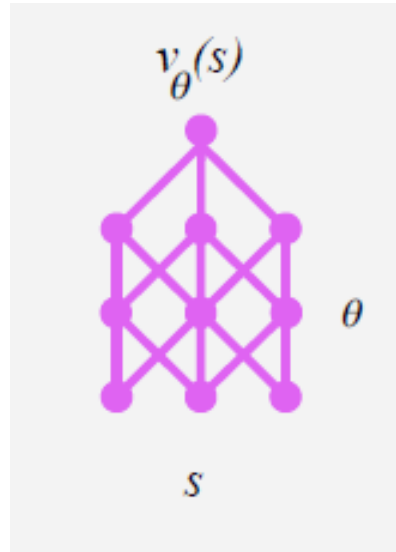
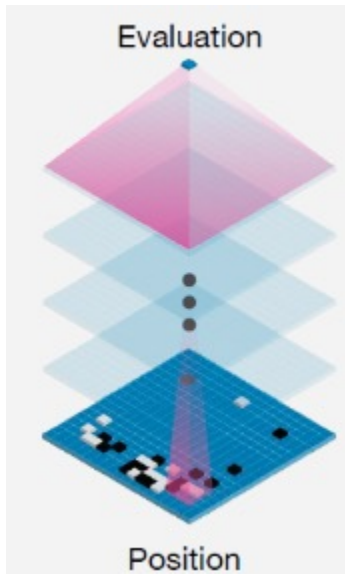
Why is Go hard for computers to play?

- Game tree complexity makes brute force search intractable



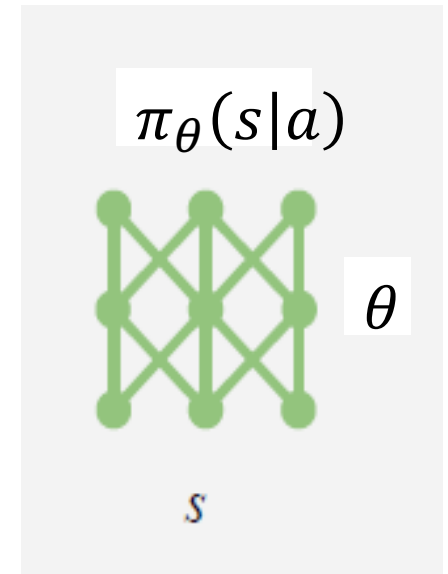
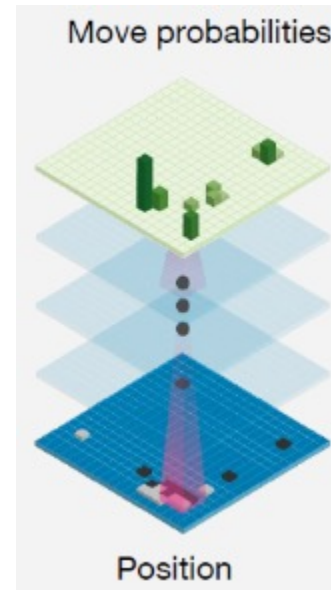
\* Based on lecture and slides of David Silver and DeepMind

# Value Network

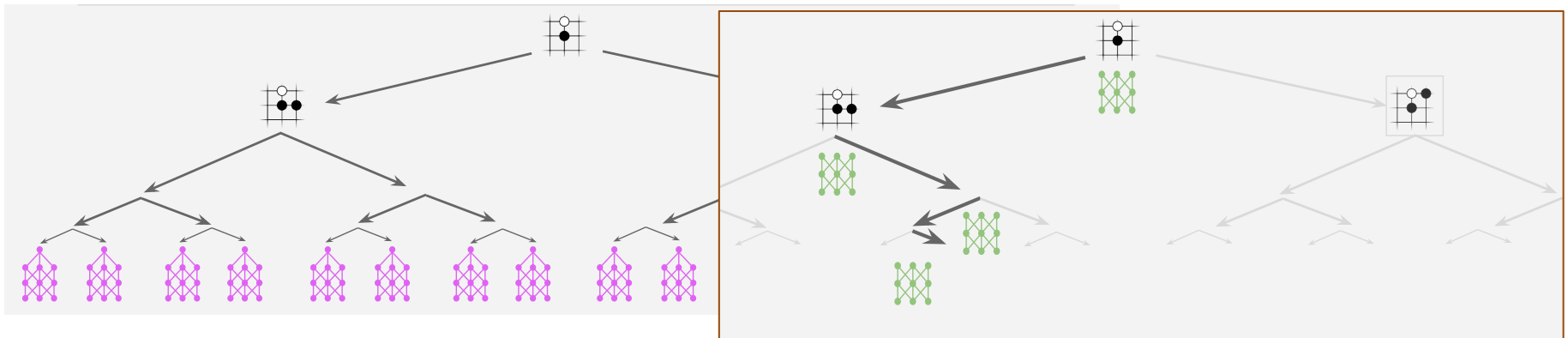


Helps to reduce depth of search

# Policy Network

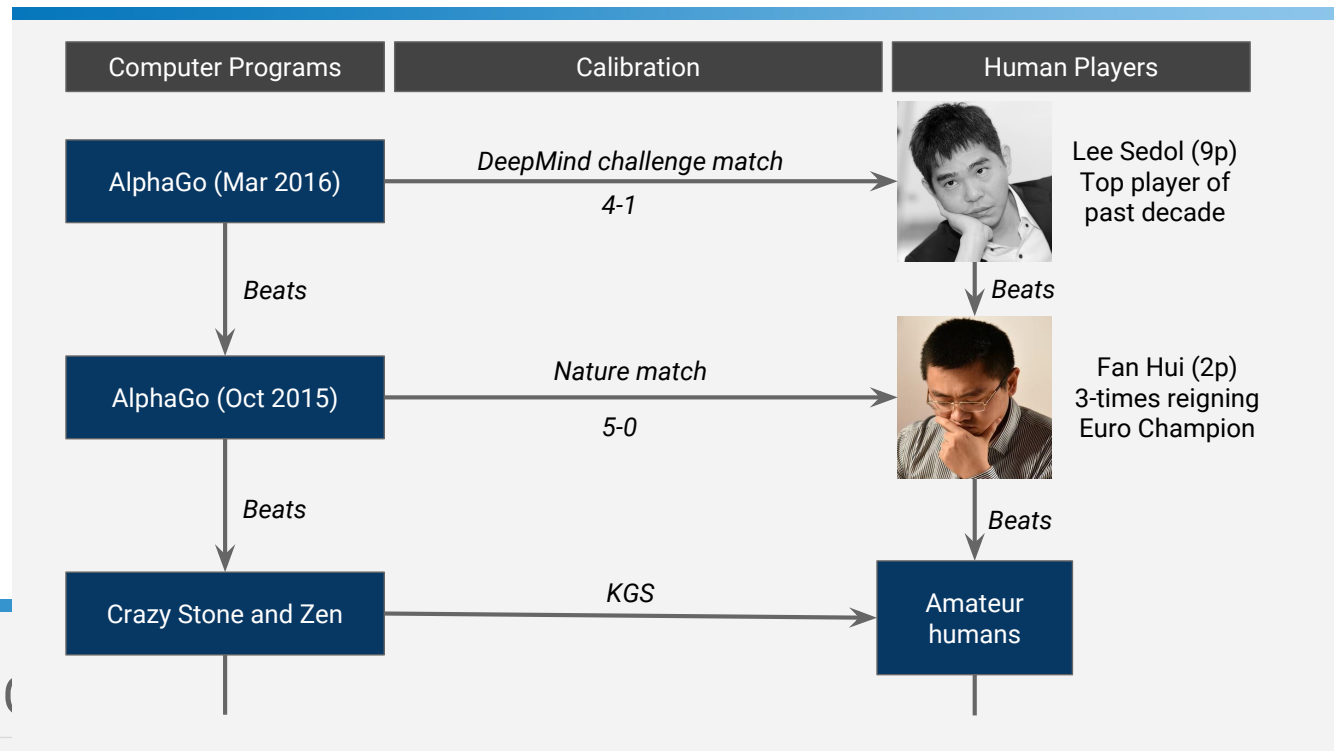


Helps to reduce breadth of search

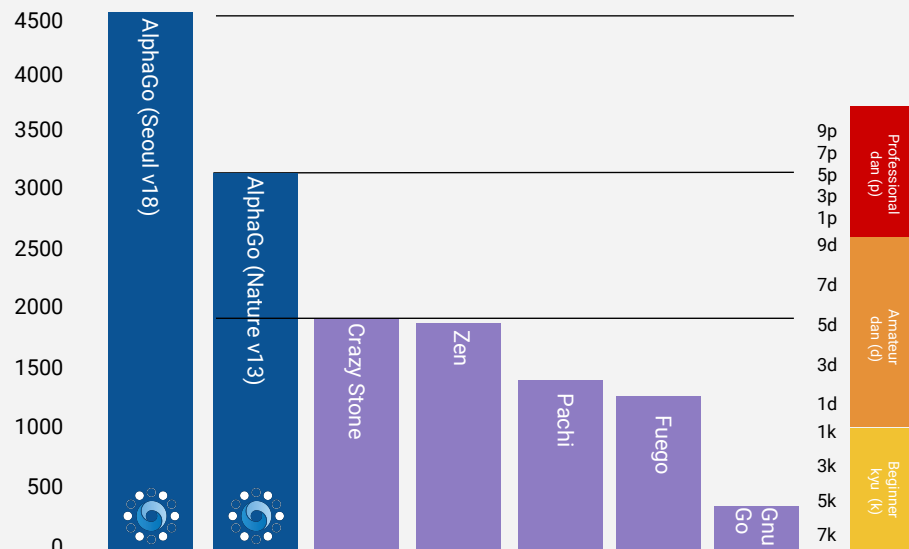


# Training

|                           | <b>Policy Network<br/>(Pre-training)</b>          | <b>Policy Network</b>                                 | <b>Value Network</b>                                 |
|---------------------------|---------------------------------------------------|-------------------------------------------------------|------------------------------------------------------|
| <b>Architecture</b>       | 12-layer CNN                                      | 12-layer CNN                                          | 12-layer CNN                                         |
| <b>Learning mode</b>      | Supervised Learning                               | Reinforcement Learning                                | Reinforcement Learning                               |
| <b>Training data</b>      | 30M positions from human expert games (5+ dan)    | Self-play                                             | Data from the Policy Network's self-play (30M games) |
| <b>Training objective</b> | Maximize likelihood by SGD (classification)       | Maximize wins                                         | Minimize MSE by SGD (regression)                     |
| <b>Training time</b>      | 4 weeks on 50 GPUs                                | 1 week on 50 GPUs                                     | 1 week on 50 GPUs                                    |
| <b>Results</b>            | 57% accuracy on test data (state-of-the-art: 44%) | Wins 80% of games against supervised learning network |                                                      |



## Evaluating AlphaGo



## ARTICLE

doi:10.1038/nature24270

# Mastering the game of Go without human knowledge

David Silver<sup>1\*</sup>, Julian Schrittwieser<sup>1\*</sup>, Karen Simonyan<sup>1\*</sup>, Ioannis Antonoglou<sup>1</sup>, Aja Huang<sup>1</sup>, Arthur Guez<sup>1</sup>, Thomas Hubert<sup>1</sup>, Lucas Baker<sup>1</sup>, Matthew Lai<sup>1</sup>, Adrian Bolton<sup>1</sup>, Yutian Chen<sup>1</sup>, Timothy Lillicrap<sup>1</sup>, Fan Hui<sup>1</sup>, Laurent Sifre<sup>1</sup>, George van den Driessche<sup>1</sup>, Thore Graepel<sup>1</sup> & Demis Hassabis<sup>1</sup>

A long-standing goal of artificial intelligence is an algorithm that learns, *tabula rasa*, superhuman proficiency in challenging domains. Recently, AlphaGo became the first program to defeat a world champion in the game of Go. The tree search in AlphaGo evaluated positions and selected moves using deep neural networks. These neural networks were trained by supervised learning from human expert moves, and by reinforcement learning from self-play. Here we introduce an algorithm based solely on reinforcement learning, without human data, guidance or domain knowledge beyond game rules. AlphaGo becomes its own teacher: a neural network is trained to predict AlphaGo's own move selections and also the winner of AlphaGo's games. This neural network improves the strength of the tree search, resulting in higher quality move selection and stronger self-play in the next iteration. Starting *tabula rasa*, our new program AlphaGo Zero achieved superhuman performance, winning 100–0 against the previously published, champion-defeating AlphaGo.

Nature, October 2017

Human expert positions



Classification

Supervised Learning policy network



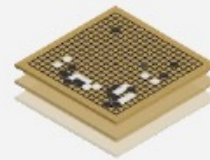
Self Play

Reinforcement Learning policy network



Self Play

Self-play data



Regression

Value network

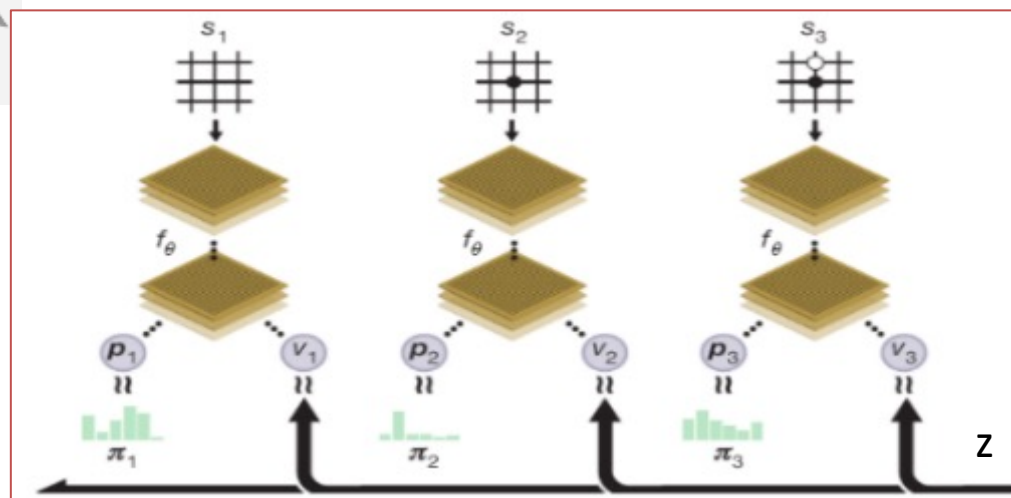


1. Is this necessary?

2. Why are the policy network and the value network trained separately?



Training



Policy network

Value network

## David Silver on **AlphaGo Zero**:

<https://www.youtube.com/watch?v=tXlM99xPQC8>

**Tabula rasa** refers to the idea that people are born without built-in mental content and therefore all the knowledge of a person comes from his/her experience or perception.

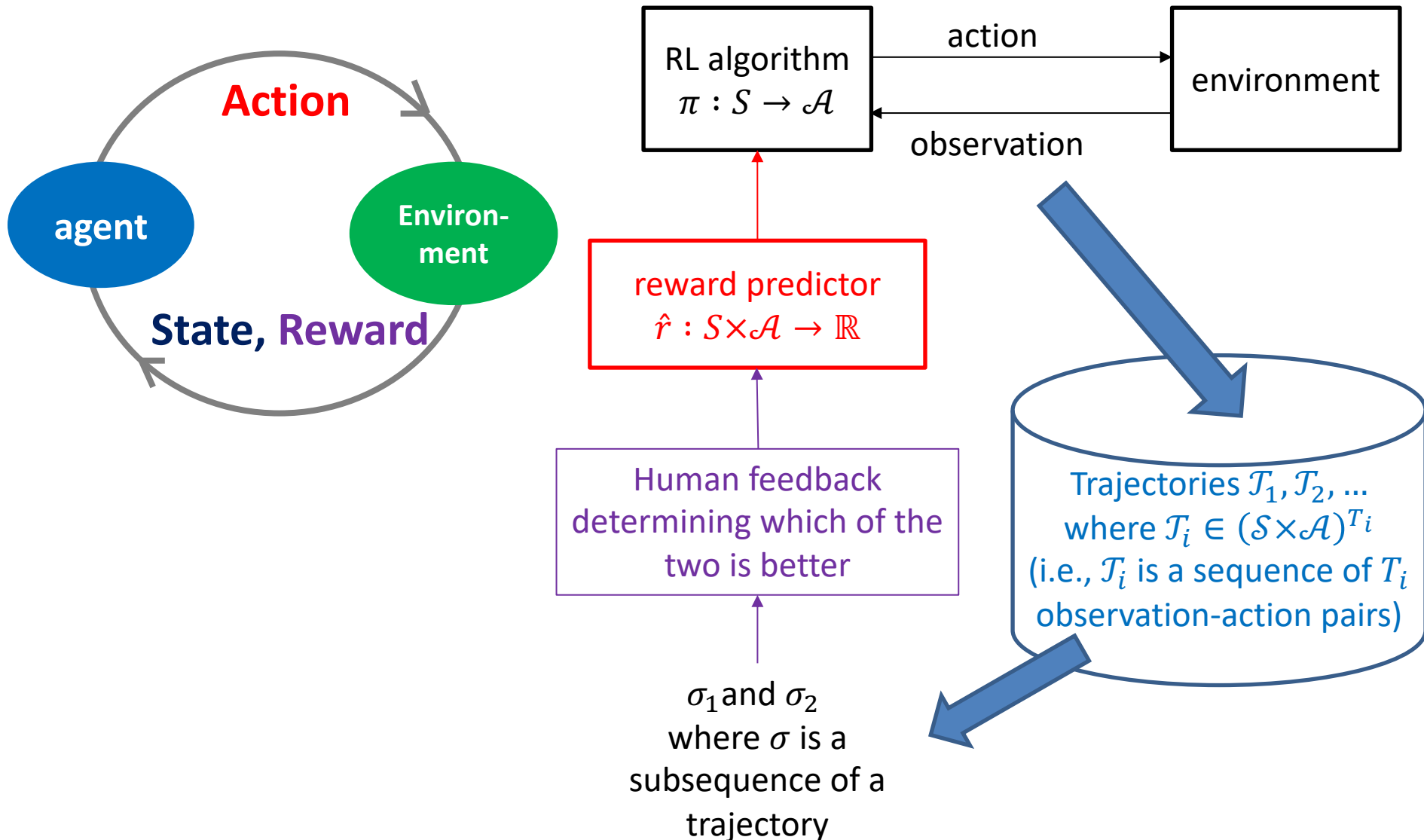
TECH \ ARTIFICIAL INTELLIGENCE \

## DeepMind's AI became a superhuman chess player in a few hours, just for fun

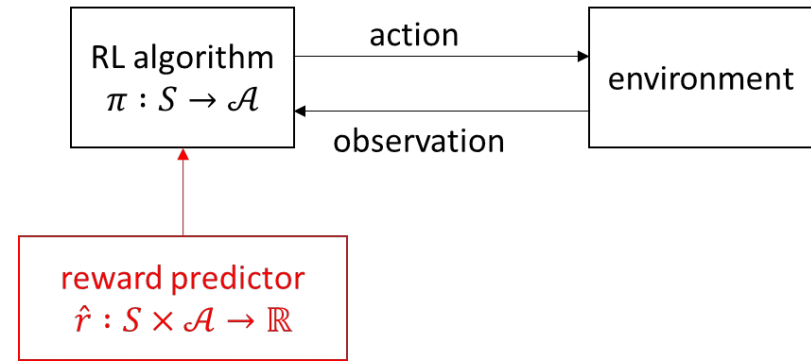
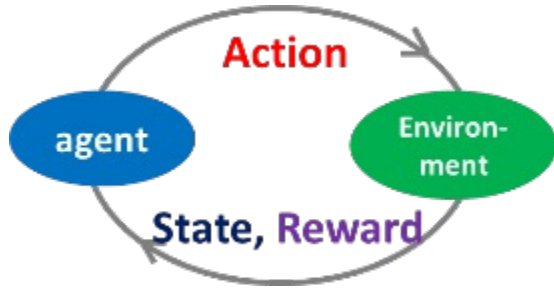
*The descendant of DeepMind's world champion Go program stretches its muscles in a new domain*

By James Vincent | @jvincent | Dec 6, 2017, 8:11am EST

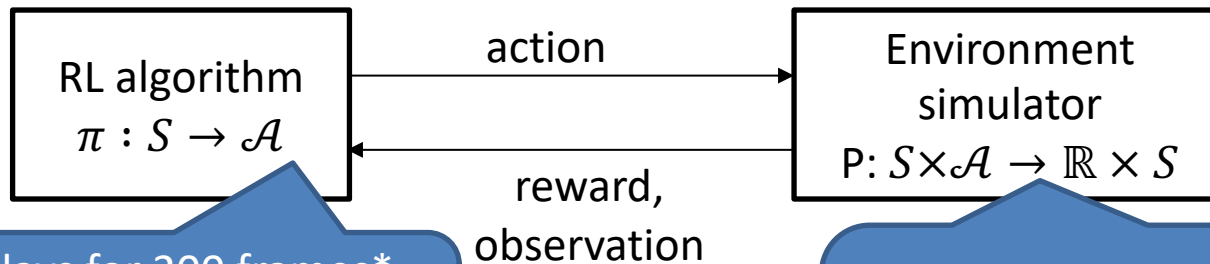
# RL systems with human feedback



# RL systems with human feedback



## Transformers are Sample Efficient World Models (2023) – Atari games



1. Agent plays for 200 frames\* to produce training data for the Environment predictor

2. With the data, transformer (which excels at predicting next item in sequence) learns to predict next frame\* (and reward)

3. After Environment simulator is trained, the Agent can learn from the simulation

*\*Frames are represented by tokens and an autoencoder learns the representation*